
MICA: Measurement, Instrumentation, Control and Analysis

Hammerstein System Identification: Quadratic

Ariel Mobius, MIT

7 May 2026

Revisions

2025	Dec 04	Updated from previous versions
	08	Changed to using SVD based Toeplitz impulse response function estimates
	09	Added more keywords

Keywords

Hammerstein System, Wiener System, NL System, LN System, Static Nonlinearity, Quadratic Nonlinearity, Linear Dynamic Subsystem, Impulse Response Function, Transfer Function, Second-Order System, Low-Pass Filter, Under-Damped System, Natural Frequency, Damping Ratio, Static Gain, Inverse Laplace Transform, Sampling Interval, Gaussian White Noise, Stochastic Input, Numerical Convolution, ListConvolve, Probability Density Function, Probability Distribution Function, Auto-Correlation Function, Cross-Correlation Function, Biased Correlation, Covariance Function, Toeplitz Matrix, Singular Value Decomposition (SVD), Pseudo-Inverse, Regularization, Singular Value Thresholding, Bussgang's Theorem, Scaling Ambiguity, Variance Accounted For (VAF), Residuals, Prediction Error, Deconvolution, Wiener Deconvolution, ListDeconvolve, Internal Signal Estimation, Cross-Plotting, Polynomial Fitting, LinearModelFit, Iterative Refinement, Non-Parametric Estimation, Parametric Model, Levenberg-Marquardt Algorithm, NonlinearModelFit, Objective Function, Model Complexity, FindFormula, Model Discovery, System Identification, Black Box Modeling, Input-Output Analysis

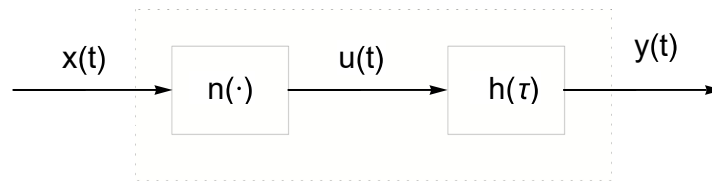
Objectives

The purpose of this MICA Module is to define Hammerstein systems and introduce techniques which under certain circumstances enables Hammerstein systems to be characterized experimentally.

Hammerstein systems

A static nonlinearity (e.g., cuber, hard limiter, etc) followed by a linear dynamic subsystem is called a Hammerstein system after the German mathematician Adolf Hammerstein (if you can read German see https://de.wikipedia.org/wiki/Adolf_Hammerstein) who studied such systems.

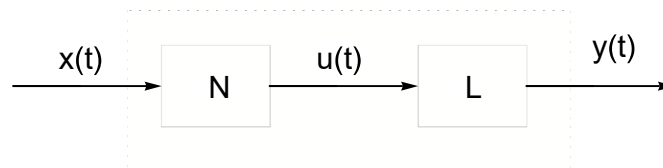
Hammerstein System



Here we see a Hammerstein system consisting of a static nonlinear element, $n(\cdot)$, followed by a linear dynamic subsystem as represented by an impulse response function, $h(\tau)$. The input to the Hammerstein system is denoted by, $x(t)$, and the output from the Hammerstein system is denoted by, $y(t)$. An internal signal $u(t)$ is the output from the static nonlinear element, $n(\cdot)$, and is also the input to the linear dynamic component, $h(\tau)$.

Sometimes Hammerstein systems are called NL systems because there is a static nonlinearity (N) followed by a linear dynamic system (L). An LN system consists of a linear dynamic element (L) followed by a nonlinear static element (N) and is called a Wiener system. Wiener system identification is the subject of another MICA Notebook.

Hammerstein System



In this MICA Notebook we will simulate a Hammerstein system and attempt to discover, $n(\cdot)$ and $h(\tau)$, purely from the input, $x(t)$, and output, $y(t)$, data sets.

Note

You can progress through this Notebook by clicking on the ■ at the start of each region of code. This will execute (evaluate) the region of code associated with ■ and any results of the code execution will be generated and added to the Notebook. After inspecting the results and reading subsequent text you can use the mouse scroll wheel (if available) to scroll down to the next ■ .

The whole Notebook can be executed by selecting Evaluation from the menu above and clicking on Evaluate Notebook. While it is OK to do this here, it can be potentially disastrous if this is done in MICA Notebooks containing code to run experiments.

Create Hammerstein system

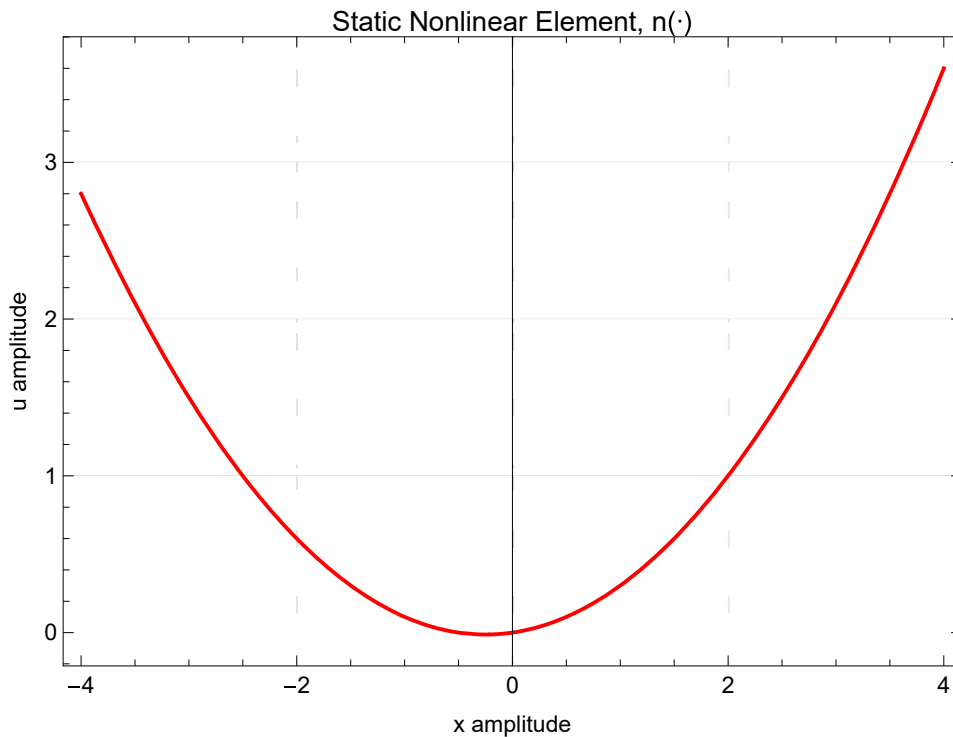
As an example lets start by making the nonlinear static element, $n(\cdot)$, a saturation nonlinearity and generate a second-order, low-pass, under-damped impulse response function, $h(\tau)$, (to model the linear dynamic element).

Saturation static nonlinearity

In this example we will make the static nonlinearity a quadratic:

```
In[ ]:= n[xx_] := 0.2 xx^2 + 0.1 xx;  
Plot[n[xx], {xx, -4, 4}, GridLines -> Automatic, PlotStyle -> Red, Frame -> True,  
LabelStyle -> {12, Black}, FrameLabel -> {"x amplitude", "u amplitude"}, AspectRatio -> 0.7,  
PlotLabel -> "Static Nonlinear Element, n(.)", PlotRange -> All, ImageSize -> 500]
```

Out[]:=



Generate $h(\tau)$

Lets choose a second – order low – pass subsystem. The general transfer function for such a sub system is :

```
In[ ]:= H[s_, α_, ω_, ξ_] := 
$$\frac{\alpha \omega^2}{s^2 + 2 \xi \omega s + \omega^2} (* *)$$

```

If we inverse Laplace transform this transfer function then we get:

```
In[ ]:= InverseLaplaceTransform[H[s, α, ω, ξ], s, τ] // FullSimplify  
Out[ ]:= 
$$\frac{e^{-\xi \tau \omega} \alpha \omega \operatorname{Sinh}\left[\sqrt{-1 + \xi^2} \tau \omega\right]}{\sqrt{-1 + \xi^2}}$$

```

This is the system continuous (or symbolic) unit impulse response function which we will denote h in order to distinguish it from the sampled version (see below) which we will denote, h . We also denote

the continuous lag domain by τ to distinguish it from the sampled version (see below) which is denoted by τ_i .

$$\text{In}[] := \blacksquare; \text{h}[\tau_ , \alpha_ , \omega_ , \xi_] := \frac{e^{-\xi \tau \omega} \alpha \omega \text{Sinh}\left[\sqrt{-1 + \xi^2} \tau \omega\right]}{\sqrt{-1 + \xi^2}}$$

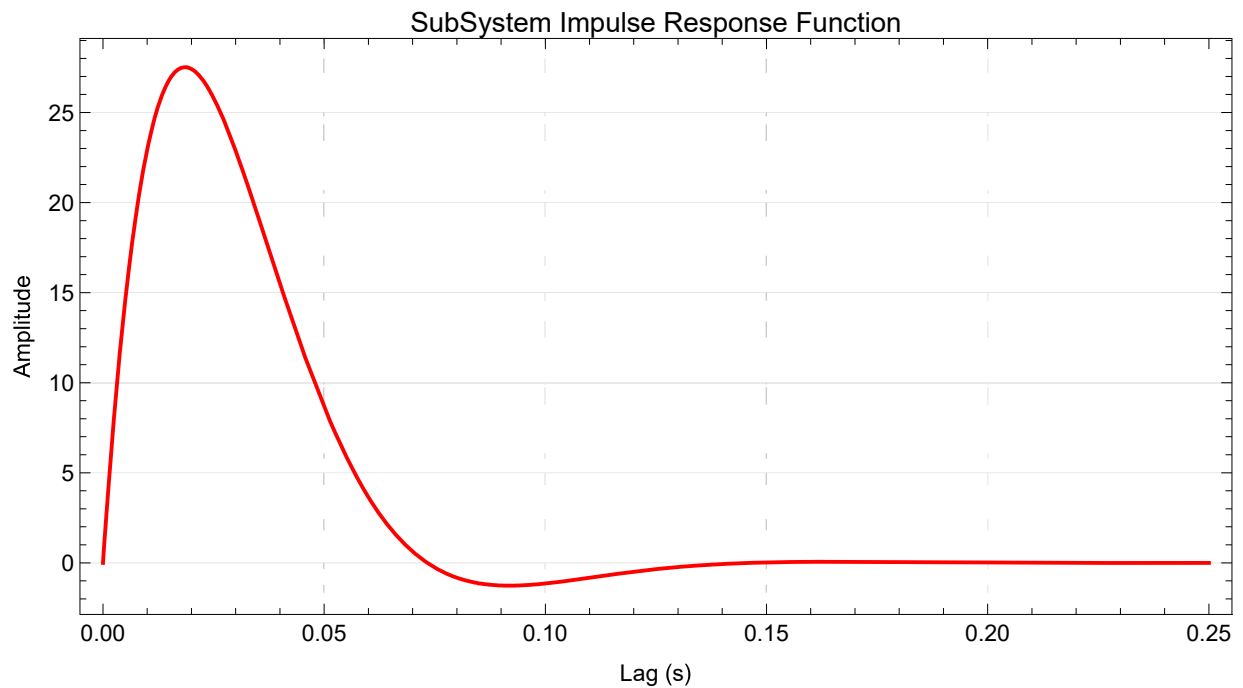
Lets make the static gain, $\alpha = 1$, the natural frequency, $\omega = 60$ rad/s (around 10 Hz), and the damping parameter $\zeta = 0.7$.

```
In[ ] := \blacksquare; h[\tau_] := h[\tau, 1, 60, 0.7] // FullSimplify // Re
```

Lets plot this:

```
In[ ] := \blacksquare; Plot[h[\tau], {\tau, 0, 0.25}, GridLines -> Automatic, PlotStyle -> Red,
  Frame -> True, LabelStyle -> {12, Black}, FrameLabel -> {"Lag (s)", "Amplitude"},
  AspectRatio -> 0.5, PlotLabel -> "SubSystem Impulse Response Function",
  PlotRange -> All, ImageSize -> Automatic -> 600]
```

Out[] =



Notice that this impulse response function “dies out” by around 0.15 s.

We will need to sample this symbolic version of the impulse response function to produce a sampled version. To do this we need to specify the inter-sample interval, $\Delta\tau$, and the maximum lag, τ_{Max} . Lets make the sampling increment be 2 ms (i.e., a sampling rate of 500 Hz) and the maximum lag be 0.25 s (as above).

```
In[ ] := \blacksquare; \Delta\tau = 0.002; \tau_{\text{Max}} = 0.25;
```

We can now generate a sampled version of the lag domain, τ :

```
In[ ] := \blacksquare; \tau_i = \text{Range}[0, \tau_{\text{Max}}, \Delta\tau];
```

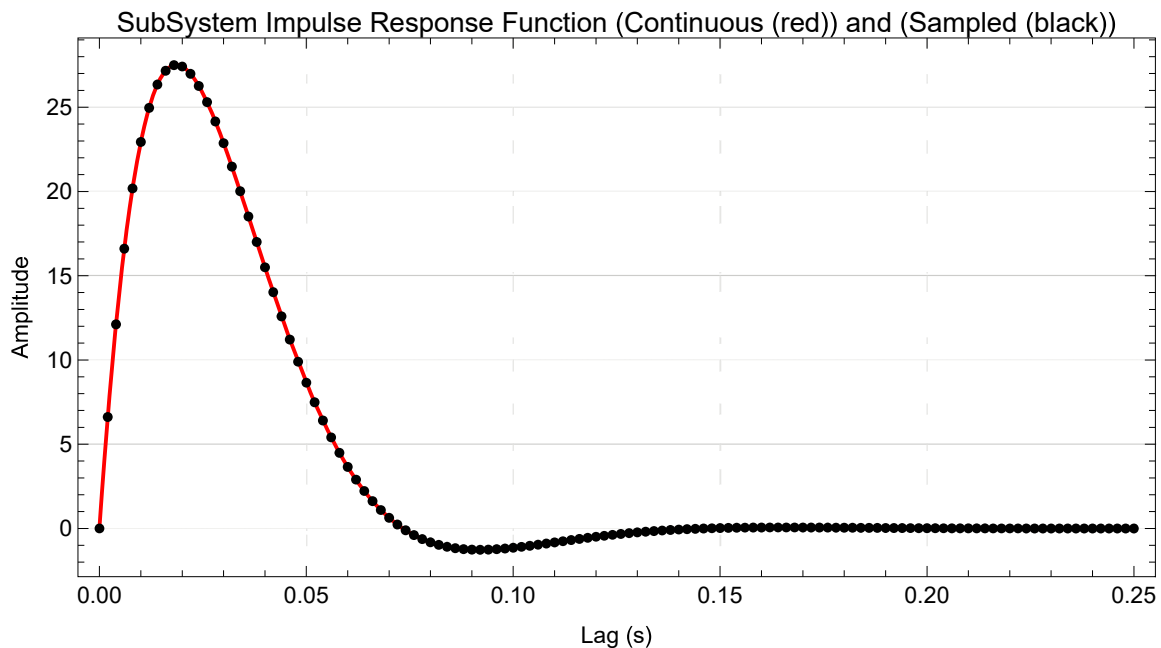
and a sampled version of the impulse response function:

```
In[ ]:= ■; hi = h[ $\tau$ i];
```

Now we can plot both the continuous and sampled versions of h:

```
In[ ]:= ■; Show[
  Plot[h[ $\tau$ ], { $\tau$ , 0, 0.25}, PlotStyle → Red,
  Frame → True, LabelStyle → {12, Black}, GridLines → Automatic,
  FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5, PlotLabel →
    "SubSystem Impulse Response Function (Continuous (red)) and (Sampled (black))",
  PlotRange → All, ImageSize → 600],
  ListPlot[{ $\tau$ i, hi}^T, PlotStyle → Black]]
```

```
Out[ ]:=
```



It is always a good idea to plot the individual points in the impulse response function to check that the sampling rate is high enough given the time course of the function. It is best to have plenty of points (e.g., > 5) covering the initial rise of the impulse response function. It is also best to have a domain (whose last value is τ_{Max}) which is $1.5 \times$ to $2 \times$ the time it takes for the impulse response to “die out”. If these conditions are not met then resample the original continuous (symbolic) impulse response function with a more appropriate sampling interval, $\Delta\tau$, and maximum lag, τ_{Max} .

Note that the area under the impulse response function should equal the static gain, which is 1.0 in this case. Lets check by crude rectangular numeric integration:

```
In[ ]:= ■; Total[hi]  $\Delta\tau$ 
```

```
Out[ ]:=
```

```
0.998832
```

This is close enough.

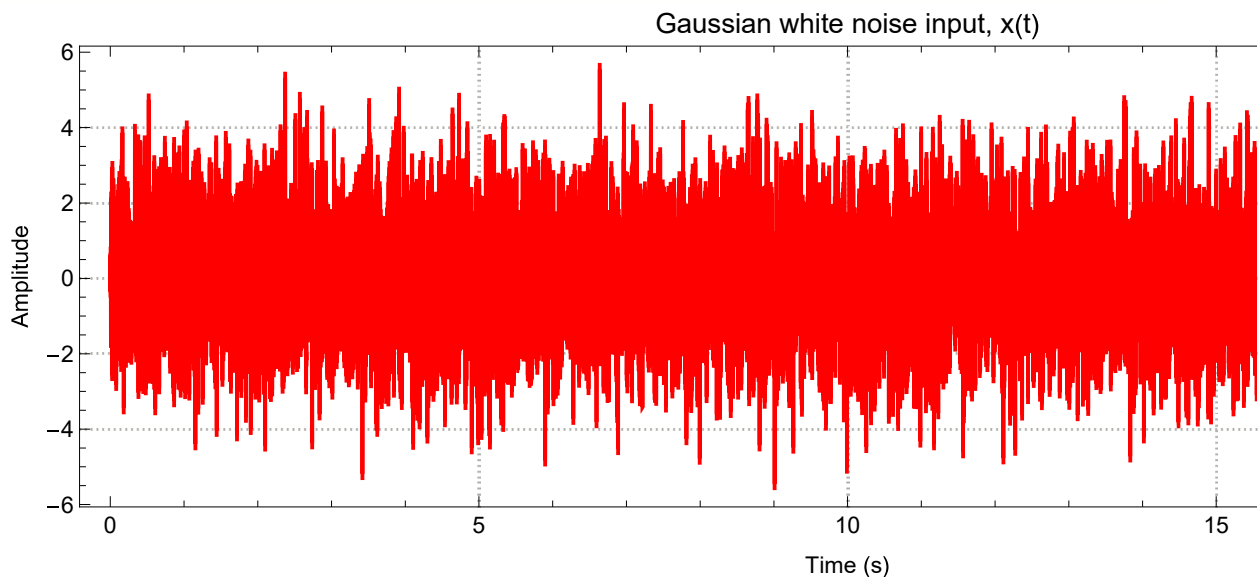
Generate the input, $x(t)$, the unobservable internal signal, $u(t)$ and the output, $y(t)$

Generate Gaussian white noise signal, $x(t)$

We start by generating 20 seconds of a Gaussian white noise input, $x(t)$, sampled at around 500 Hz, with a mean of 0 and a standard deviation of 1. Note that we could have used pretty much any stochastic input here except a stochastic binary input (which is not sufficient to identify a nonlinear system). Note that as before we generate a sampled version of the time domain and specify a maximum time, t_{Max} . Note also that we use the same sampling interval in time, Δt , as we used for lag, $\Delta \tau$.

```
In[ ]:= ■; tMax = 20.0;  
Δt = Δτ;  
ti = Range[0.0, tMax, Δt];  
x = Table[RandomVariate[NormalDistribution[0, 1.5]], Length[ti]];  
ListLinePlot[{ti, x}^T, PlotTheme → "Detailed",  
PlotRange → All, PlotStyle → Red, Frame → True, LabelStyle → {12, Black},  
FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,  
PlotLabel → "Gaussian white noise input, x(t)", ImageSize → Automatic → 800]
```

Out[]:=



Pass the input, $x(t)$, through the static nonlinear element, $n(\cdot)$, to generate the internal signal, $u(t)$

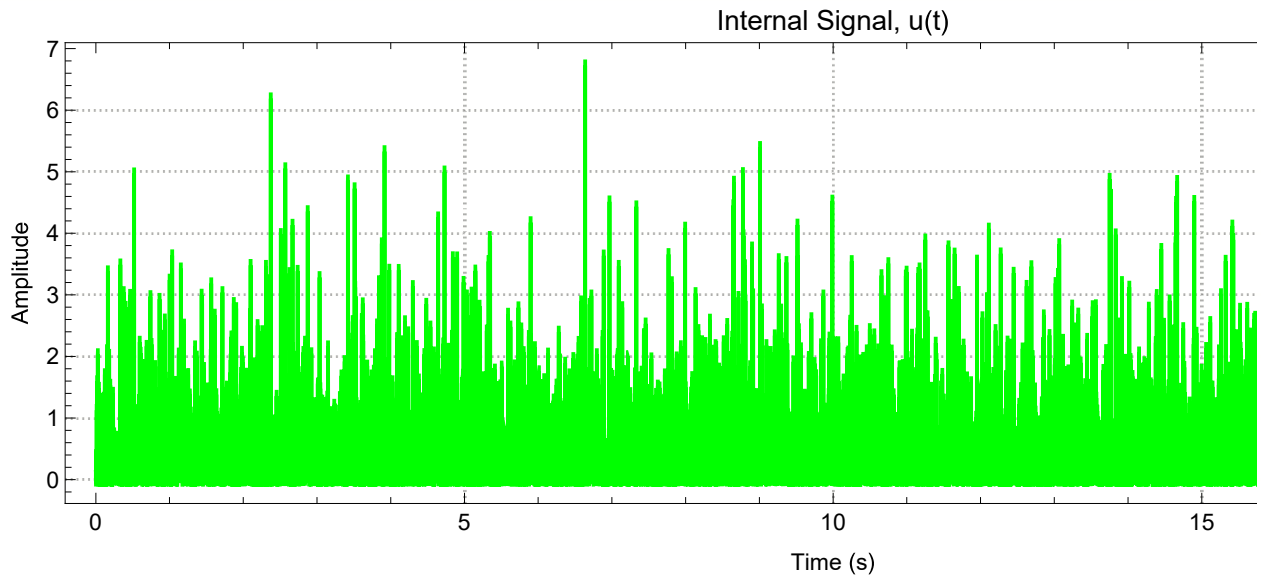
We now pass $x(t)$ through our static nonlinearity, $n(\cdot)$, to produce the internal signal, $u(t)$:

```

In[ ]:= ■; u = n[x];
ListLinePlot[{ti, u}^T, PlotTheme → "Detailed", PlotRange → All, PlotStyle → Blue,
  Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Time (s)", "Amplitude"},
  AspectRatio → 0.3, PlotLabel → "Internal Signal, u(t)", ImageSize → Automatic → 800]

```

Out[]=



Convolve, $h(\tau)$, with $u(t)$ to produce, the output, $y(t)$

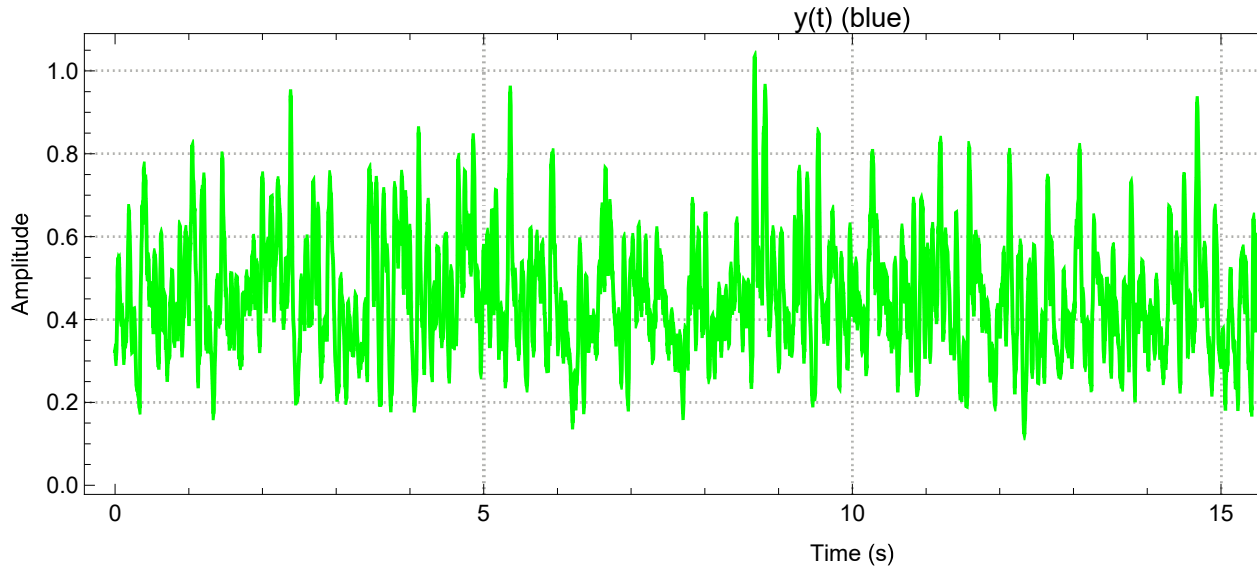
We now numerically convolve our sampled input, $x(t)$, with the sampled version of our impulse response function, $h(\tau)$, to generate the internal signal, $u(t)$:

```

In[ ]:= ■; y = Δt ListConvolve[hi, u, {1, 1}] ;
ListLinePlot[{ti, y}^T, PlotTheme → "Detailed", PlotRange → All, PlotStyle → Blue,
  Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Time (s)", "Amplitude"},
  AspectRatio → 0.3, PlotLabel → "y(t) (blue)", ImageSize → Automatic → 800]

```

Out[]=



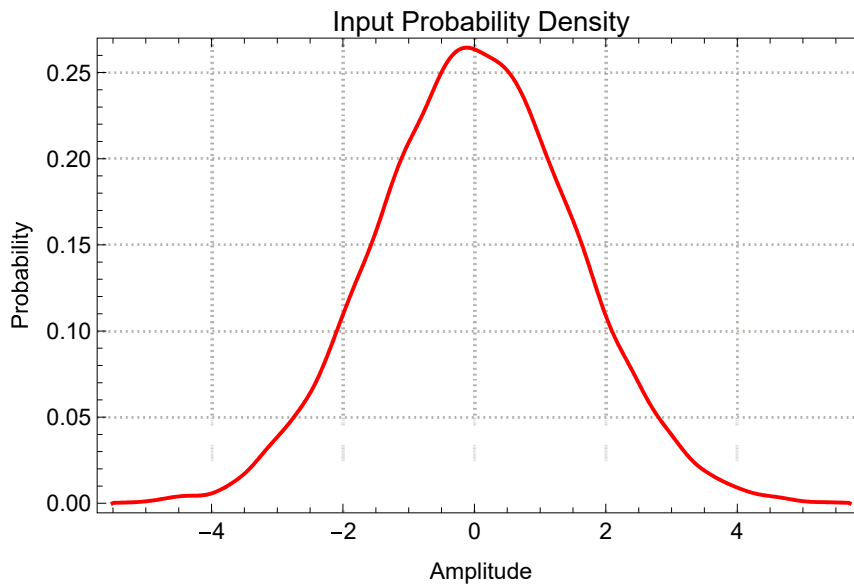
Compare input and output probability density functions

We now calculate the input and output probability density functions using Mathematica's `SmoothHistogram[]` function which determines the probability density functions via a different technique than the usual `Histogram[]` function. There is a whole area of statistics devoted to different probability density function determination methods.

Notice that while the original input, $x(t)$ is Gaussian:


```
In[ ]:= ▣; SmoothHistogram[x, Automatic, "PDF", PlotRange → All, PlotTheme → "Detailed",
  FrameLabel → {"Amplitude", "Probability"}, LabelStyle → {12, Black}, PlotStyle → Red,
  PlotLabel → "Input Probability Density", ImageSize → Automatic → 400]
```

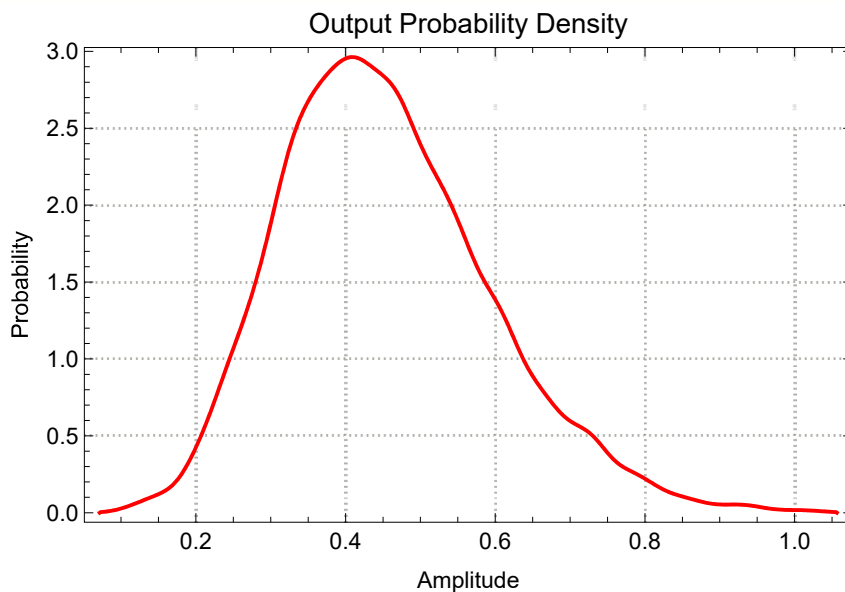
```
Out[ ]:=
```



The output, $y(t)$, is a little “less” Gaussian:

```
In[ ]:= ▣; SmoothHistogram[y, Automatic, "PDF", PlotRange → All, PlotTheme → "Detailed",
  FrameLabel → {"Amplitude", "Probability"}, LabelStyle → {12, Black}, PlotStyle → Red,
  PlotLabel → "Output Probability Density", ImageSize → Automatic → 400]
```

```
Out[ ]:=
```



A linear system perturbed by a Gaussian stochastic input (does not need to be white) will produce a Gaussian stochastic output. If the output from a system driven by a Gaussian stochastic input is non Gaussian then this is clear evidence that the system is nonlinear. The output probability density func-

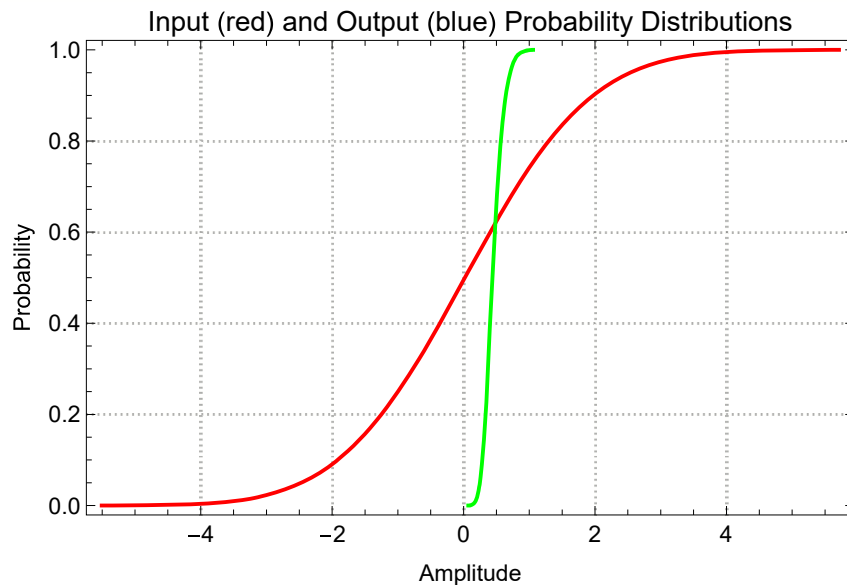
tion looks Gaussian so we cannot at this point conclude that we have a nonlinear system. Remember to distinguish a necessary condition from a sufficient condition: if the input is Gaussian and the output is Gaussian it could still be a nonlinear system.

Compare input and output probability distribution functions

Lets calculate the smoothed input and output probability distribution functions:

```
In[ ]:= ■; Show[
  SmoothHistogram[x, Automatic, "CDF", PlotStyle → Red, PlotRange → All, PlotTheme → "Detailed",
    FrameLabel → {"Amplitude", "Probability"}, LabelStyle → {12, Black},
    PlotLabel → "Input (red) and Output (blue) Probability Distributions",
    ImageSize → Automatic → 400],
  SmoothHistogram[y, Automatic, "CDF", PlotStyle → Blue]]
```

Out[]:=



Hammerstein system identification

We now ask if it is possible to determine the two internal subsystems, namely, $h(\tau)$ and $n(\cdot)$ solely from the input, $x(t)$, and output, $y(t)$, signals.

Lets see what happens when we compute the nonparametric impulse response function, $h_1(\tau)$, between the input, $x(t)$, and the output, $y(t)$.

To do this we compute the input auto-correlation function, $xacf(\tau)$, and the cross-correlation function between $x(t)$ and $y(t)$, $xyccf(\tau)$.

Define a function to perform biased auto- and cross-correlation functions (strictly speaking it calculates auto- and cross-covariance functions)

```
In[ ]:= ■; cor[x_, y_, m_] := With[{n = Length[x], xMn = Mean[x], yMn = Mean[y]},
  Table[(x[[1 ;; n - j]] - xMn) . (y[[1 + j ;; n]] - yMn) / n, {j, 0, m}]]
```

```

In[ ]:= ■; xacf = cor[x, x,  $\tau_{\text{Max}} / \Delta t$ ];
        xyccf = cor[x, y,  $\tau_{\text{Max}} / \Delta t$ ];

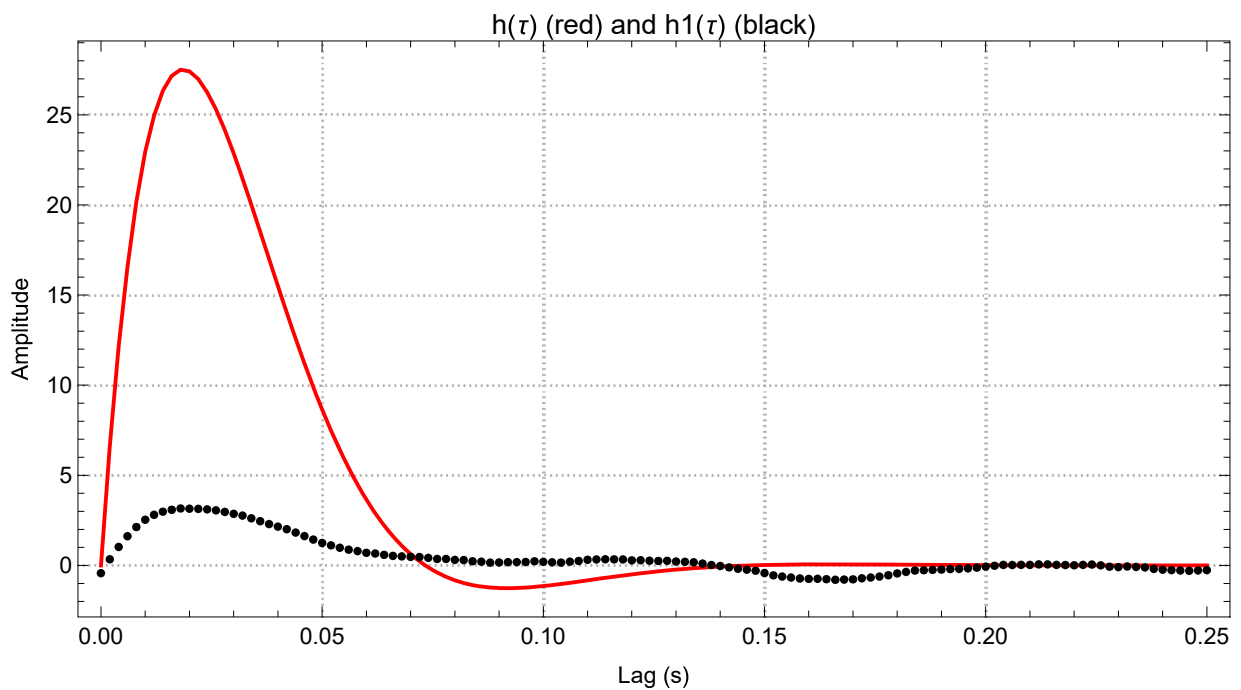
        (*toe=ToeplitzMatrix[xacf];
        invtoe=Inverse[toe];
        hi1=(invtoe.xyccf)/ $\Delta t$ ;*)

        {U, S, V} = SingularValueDecomposition[ToeplitzMatrix[xacf]];
        threshold = 0.1 Max[S]; (*Keep only significant singular values*)
        Sinv = DiagonalMatrix[1 / Max[], threshold] & /@ Diagonal[S];
        hi1 =  $\frac{1}{\Delta t}$  V.Sinv.Transpose[U].xyccf;

ListPlot[{{ $\tau_i$ , hi}T, { $\tau_i$ , hi1}T}, Joined → {True, False},
        PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
        LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5,
        PlotLabel → "h( $\tau$ ) (red) and h1( $\tau$ ) (black)", PlotRange → All, ImageSize → Automatic → 600]

```

Out[]=



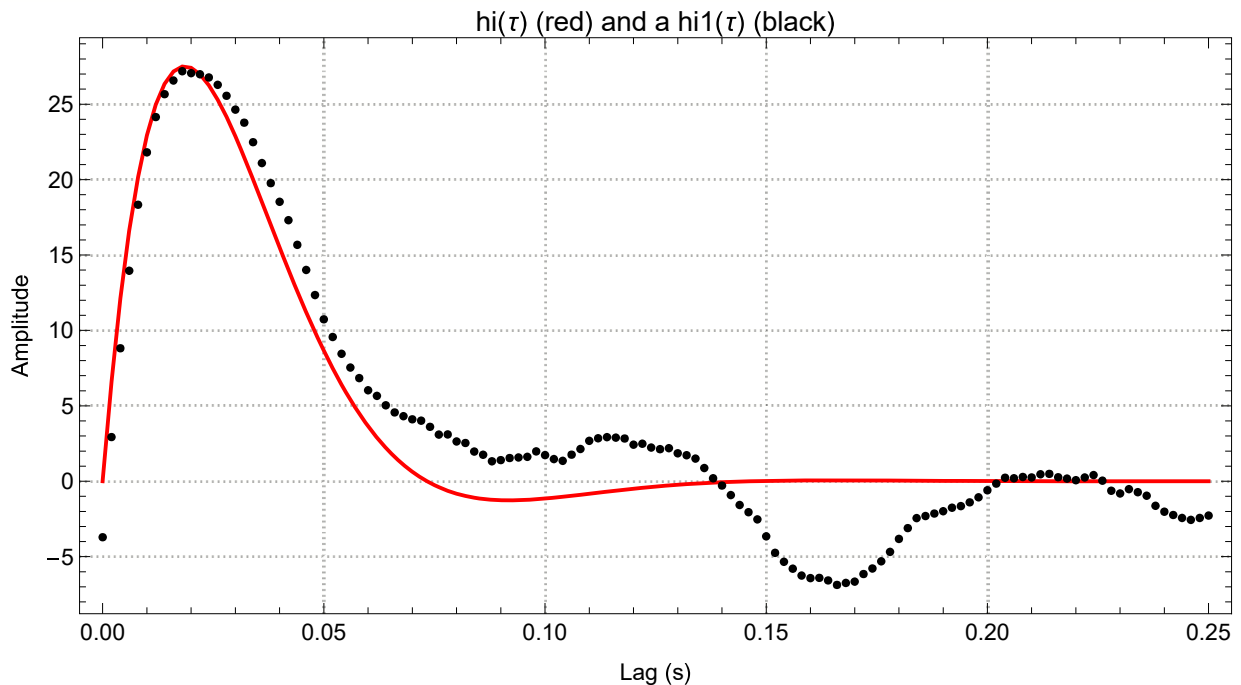
If we multiply $h(\tau)$ by α and divide $n(\cdot)$ by α there will be no change in the output, $y(t)$. In other words there is a scaling constant, α , that can freely move from $h(\tau)$ to $n(\cdot)$.

```

In[ ]:= ■; ListPlot[{{τi, hi}^T, {τi, hi1 / (Δτ Total[hi1])}^T},
  Joined → {True, False}, PlotTheme → "Detailed", PlotStyle → {Red, Black},
  Frame → True, LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"},
  AspectRatio → 0.5, PlotLabel → "hi(τ) (red) and a hi1(τ) (black)",
  PlotRange → All, ImageSize → Automatic → 600]

```

Out[]:=



This is a remarkable result!! It shows that despite the presence of a major static nonlinearity such as a cuber, the stochastic system identification techniques were able to nevertheless get an estimate of the Hammerstein system's dynamic linear element. This amazing result (for the case of cross-correlation) seems to have been first discovered by Julian Bussgang (https://en.wikipedia.org/wiki/Julian_J._Bussgang) in 1951 during his Master's thesis research when he was a student at the Research Lab for Electronics (RLE) at MIT (https://en.wikipedia.org/wiki/Bussgang_theorem).

Predict output, $y1(t)$

We can check to see how much of the output variance we can predict simply using our estimate of the impulse response function:

```

In[ ]:= ■; y1 = Δt ListConvolve[hi1, x, {1, 1}] ;

```

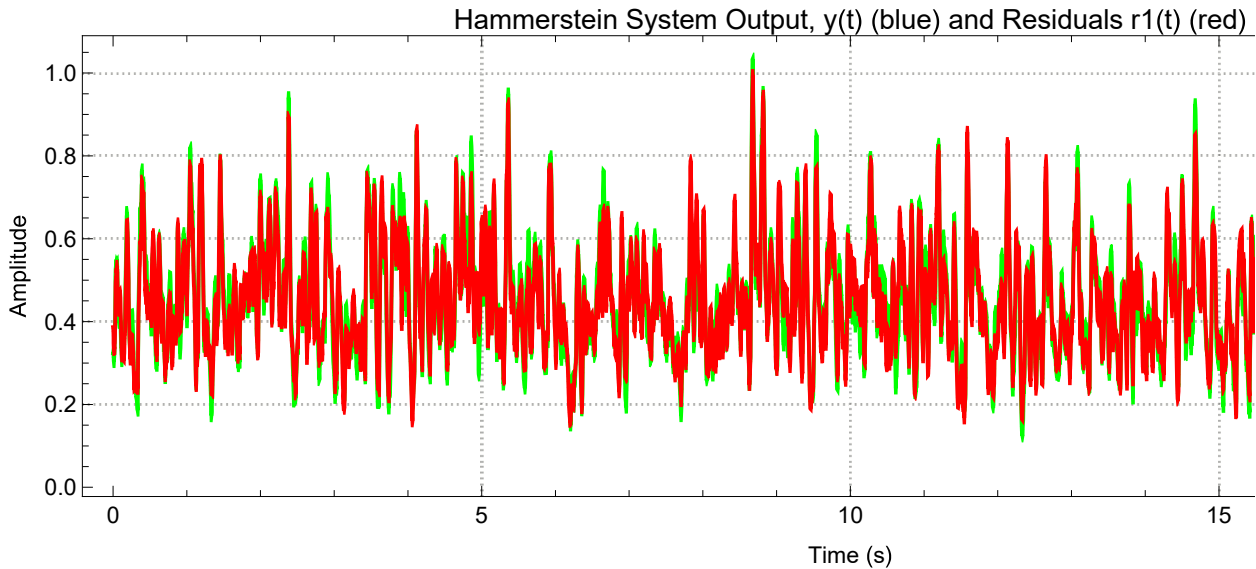
Compute the residuals

```

In[ ]:= ■; r1 = y - y1;
ListLinePlot[{{ti, y}^T, {ti, r1}^T}, PlotTheme → "Detailed",
  PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
  FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
  PlotLabel → "Hammerstein System Output, y(t) (blue) and Residuals r1(t) (red)",
  ImageSize → Automatic → 800]

```

Out[]:=



```

In[ ]:= ■; vaf1 = 100.0 (1 - Variance[r1] / Variance[y])

```

Out[]:=

7.48187

This VAF is low. The remaining variance not accounted for by our linear dynamic model is a mixture of noise and or nonlinearities. At this point we could assume that the system is linear and proceed to find a suitable parametric model. Alternatively we could decide that it is worth exploring the possibility that the system is nonlinear. We might assume that the system is a Wiener system and proceed to carry out Wiener system ID or we could assume that it might be a Hammerstein system and proceed as follows:

Predict $u(t)$

Now that we have $hi1(\tau)$, if we could find its dynamic inverse, $hi1Inv(\tau)$, then we could convolve $hi1Inv(\tau)$ with the output, $y(t)$, to produce a prediction, $u1(t)$, of the internal signal, $u(t)$. Stated another way, we would like to deconvolve the impulse response function from the output, $y(t)$, to yield an estimate of $u(t)$.

Deconvolution of $hi1(\tau)$ from the output, $y(t)$, to predict the internal signal, $u(t)$

Mathematica contains some powerful deconvolution methods. They are usually used to reduce blur in images but they work for signals also:

```
In[ ]:= ▯; u1a =  $\frac{1}{\Delta\tau}$  ListDeconvolve[hi1, y, Method → "Wiener"];
```

ListDeconvolve[] produces u1a(t) shifted by half the length of hi1 plus 1. So we need to trim this first part off and add to the end:

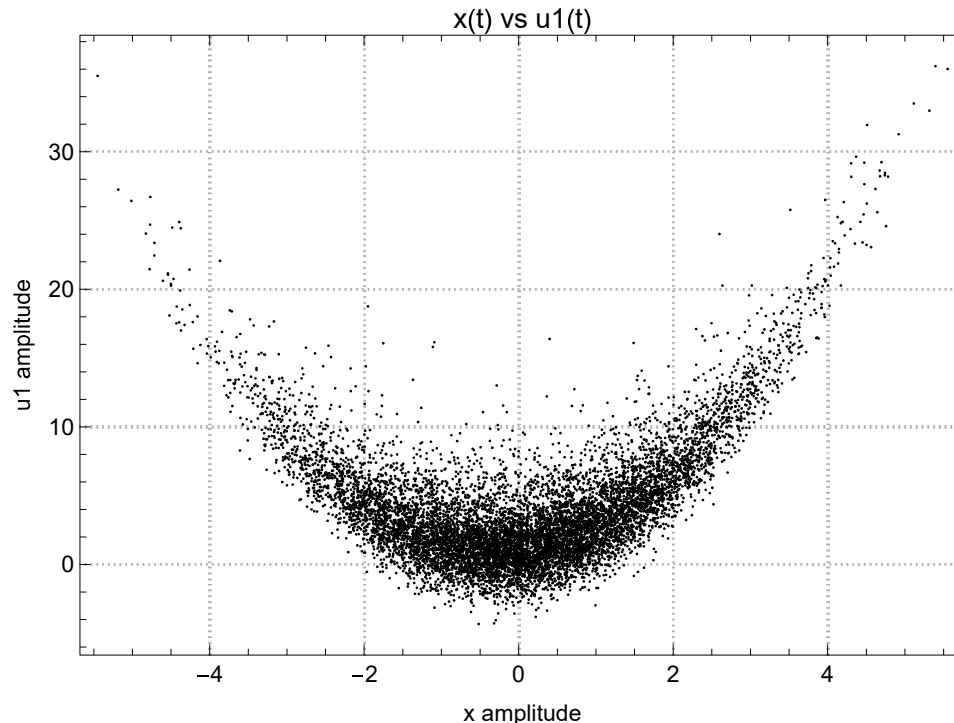
```
In[ ]:= ▯; nn = Length[x]; m = Length[hi1];
u1 = Join[u1a[[1 + (m / 2) ;; nn]], u1a[[1 ;; m / 2]]];
```

Predict n(·)

Now that we have an estimate for u1(t) we can predict the form of the static nonlinearity by simply cross-plotting x vs u1 :

```
In[ ]:= ▯; ListPlot[{x, u1}ᵀ, PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
LabelStyle → {12, Black}, FrameLabel → {"x amplitude", "u1 amplitude"},
AspectRatio → 0.7, PlotLabel → "x(t) vs u1(t)", PlotRange → All, ImageSize → 500]
```

Out[]:=



This looks like a classic saturation nonlinearity. So let's estimate the four parameters of a saturation nonlinearity model using our trusty Levenberg-Marquardt nonlinear minimization technique to fit to the x(t) vs u1(t) cross-plot after we give some initial estimates for our 4 parameters by inspection of the plot above:

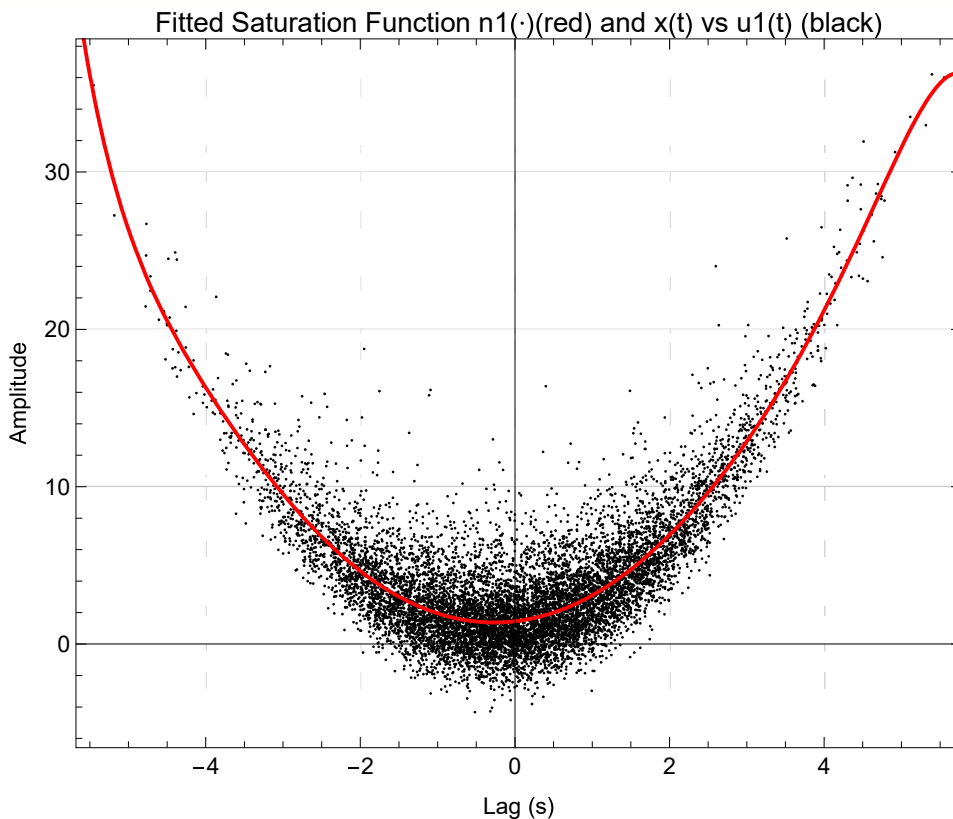
We can find an approximation to the estimate, n1(·), of n(·) above by fitting a high-order polynomial to the x(t) vs u1(t) cross-plot:

```
In[ ]:= ▯;
n1 = LinearModelFit[{x, u1}ᵀ, {1, xx, xx², xx³, xx⁴, xx⁵, xx⁶, xx⁷, xx⁸, xx⁹}, xx] ["Function"];
```

Lets plot this estimate, $n1(\cdot)$, of the nonlinearity together with the cross-plot of $x(t)$ vs $u1(t)$:

```
In[ ]:= ■; Show[
ListPlot[{x, u1}^T, PlotStyle → Black, PlotRange → All, Frame → True, PlotStyle → Red,
LabelStyle → {12, Black}, GridLines → Automatic, FrameLabel → {"Lag (s)", "Amplitude"},
PlotLabel → "Fitted Saturation Function n1(·) (red) and x(t) vs u1(t) (black)",
PerformanceGoal → "Quality", ImageSize → 500,
AspectRatio → 0.8, PerformanceGoal → "Quality"],
Plot[n1[xx], {xx, -6, 6}, PlotRange → All, PlotStyle → Red]]
```

Out[]:=



The spread in the points about the fitted model (red) is large but lets carry on in any case.

Re-predict internal signal, $u2(t)$

Now that we have the estimate, $n1(\cdot)$, we can transform the input, $x(t)$, through it to re-predict the internal signal, $u2(t)$:

```
In[ ]:= ■; u2 = n1[x];
```

Re-estimate the impulse response function

Now that we have a new prediction of the internal signal, $u2(t)$, we can re-estimate the impulse response function but now between $u2(t)$ and $y(t)$:

```

In[ ]:= ■; uacf = cor[u2, u2,  $\tau_{\text{Max}} / \Delta t$ ];
uyccf = cor[u2, y,  $\tau_{\text{Max}} / \Delta t$ ];

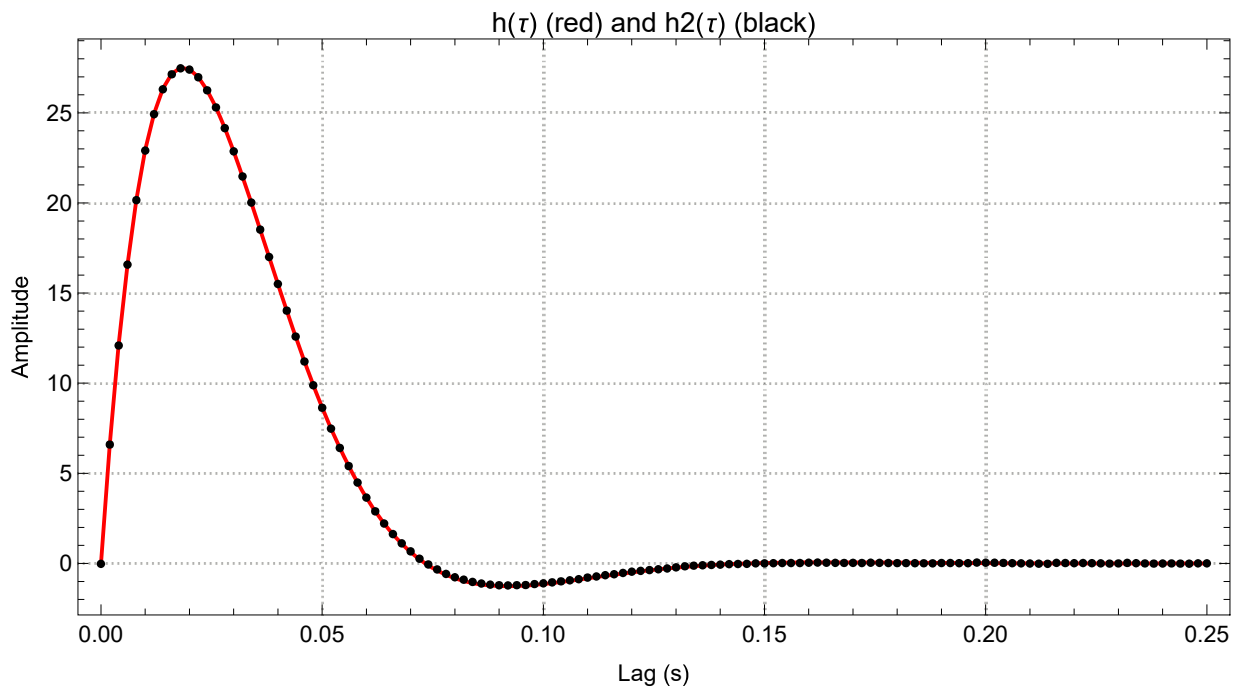
(*toe=ToeplitzMatrix[uacf];
invtoe=Inverse[toe];
hi2=(invtoe.uyccf)/ $\Delta t$ ;*)

{U, S, V} = SingularValueDecomposition[ToeplitzMatrix[uacf]];
threshold = 0.1 Max[S]; (*Keep only significant singular values*)
Sinv = DiagonalMatrix[1 / Max[#, threshold] & /@ Diagonal[S]];
hi2 =  $\frac{1}{\Delta t}$  V.Sinv.Transpose[U].uyccf;

ListPlot[{{ $\tau_i$ , hi}T, { $\tau_i$ , hi2 / ( $\Delta \tau$  Total[hi2])T}}, Joined → {True, False},
PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5,
PlotLabel → "h( $\tau$ ) (red) and h2( $\tau$ ) (black)", PlotRange → All, ImageSize → Automatic → 600]

```

Out[]:=



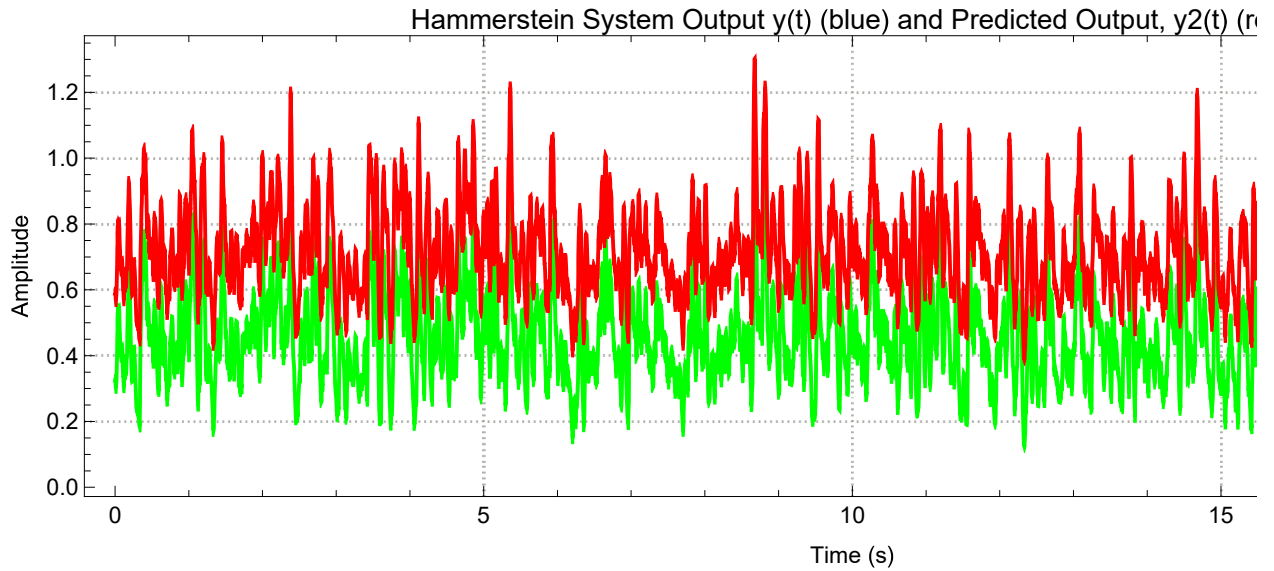
And we can now generate a new output prediction:


```

In[ ]:= ■; y2 = Δt ListConvolve[hi2, u2, {1, 1}] ;
ListLinePlot[{{ti, y}^T, {ti, y2}^T}, PlotTheme → "Detailed",
  PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
  FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
  PlotLabel → "Hammerstein System Output y(t) (blue) and Predicted Output, y2(t) (red)",
  ImageSize → Automatic → 800]

```

Out[]:=



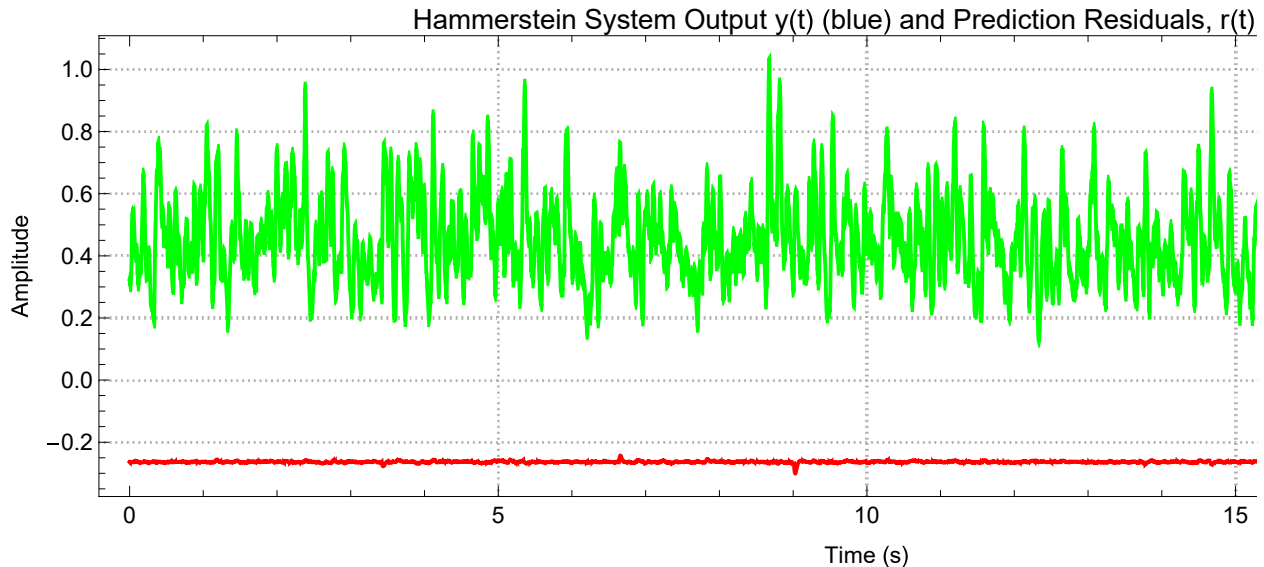
The prediction looks ok, so lets calculate the residuals, $r_2(t)$:

```

In[ ]:= ■; r2 = y - y2;
ListLinePlot[{{ti, y}^T, {ti, r2}^T}, PlotTheme → "Detailed",
  PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
  FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
  PlotLabel → "Hammerstein System Output y(t) (blue) and Prediction Residuals, r(t) (red)",
  ImageSize → Automatic → 800]

```

Out[]:=



```

In[ ]:= ■; vaf2 = 100.0 (1 - Variance[r2] / Variance[y])

```

Out[]:=

99.9648

This is dramatically higher than our earlier VAF, so we are making very good progress:

```

In[ ]:= ■; vaf1

```

Out[]:=

7.48187

However if we can assume that the noise levels are really low we might be able to do even better than this. We will therefore attempt to refine our estimates of $h(\tau)$ and $n(\cdot)$ to see if we can achieve an even higher VAF.

Redo the deconvolution of $hi2()$ from $y(t)$ to yield a new estimate of $u(t)$

```

In[ ]:= ■; u3a =  $\frac{1}{\Delta\tau}$  ListDeconvolve[hi2 / ( $\Delta\tau$  Total[hi2]), y, Method → "Wiener"];

```

The Method used in ListDeconvolve[] is based on a method developed by Norbert Wiener. But this method has nothing to do with the use of Wiener's name in association with LN systems.

Remember that ListDeconvolve[] produces $u3a(t)$ shifted by half the length of $hi2$ plus 1. So we need to trim this first part off and add to the end:

```

In[ ]:= ■; u3 = Join[u3a[[1 + (m / 2) ;; nn], u3a[[1 ;; m / 2]]];

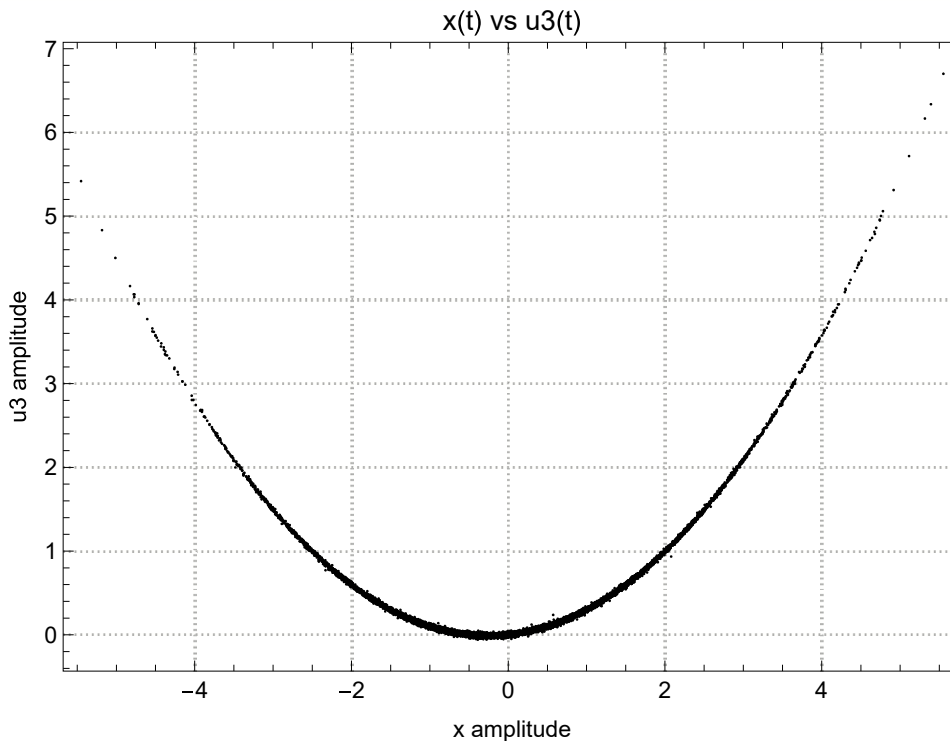
```

Re-predict $n(\cdot)$

Now that we have a new estimate, $u_3(t)$, we can predict the form of the static nonlinearity by simply cross-plotting x vs u_3 :

```
In[ ]:= ■; ListPlot[{x, u3}^T, PlotTheme -> "Detailed", PlotStyle -> {Red, Black}, Frame -> True,
  LabelStyle -> {12, Black}, FrameLabel -> {"x amplitude", "u3 amplitude"},
  AspectRatio -> 0.7, PlotLabel -> "x(t) vs u3(t)", PlotRange -> All, ImageSize -> 500]
```

Out[]:=



Now we have much less spread in the points.

So let's re-estimate the four parameters of a saturation nonlinearity model using Levenberg - Marquardt nonlinear minimization technique to fit to the $x(t)$ vs $u_3(t)$ cross-plot above to give a new estimate $n_2(\cdot)$:

```
In[ ]:= ■;
n2 = LinearModelFit[{x, u3}^T, {1, xx, xx^2, xx^3, xx^4, xx^5, xx^6, xx^7, xx^8, xx^9}, xx] ["Function"];
```

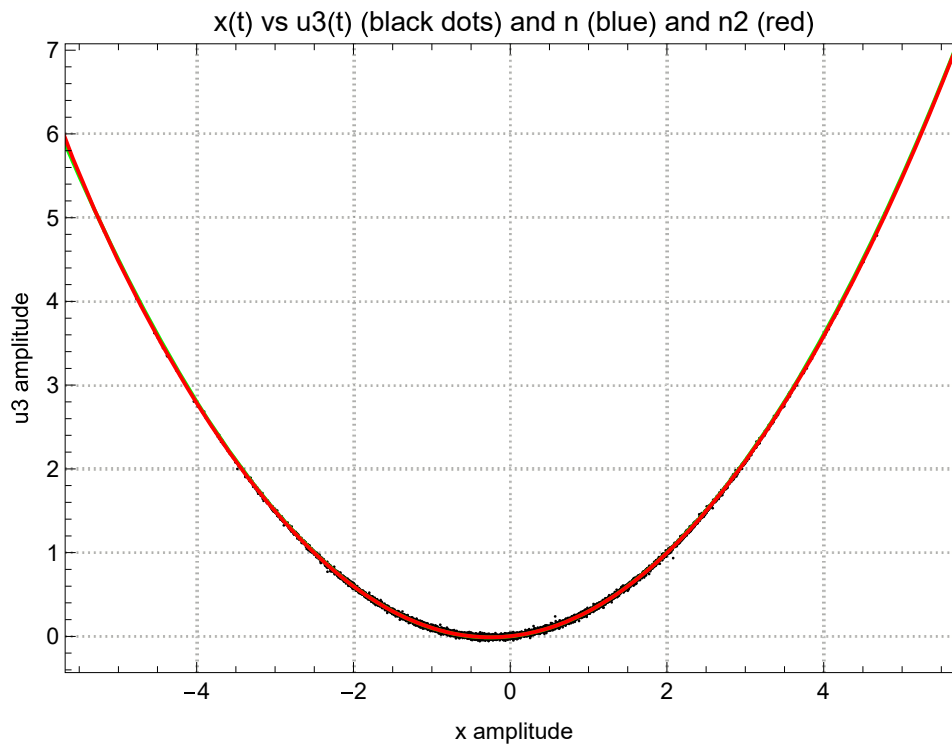
Let's plot this estimate, $n_2(\cdot)$, of the nonlinearity together with the cross-plot of $x(t)$ vs $u_3(t)$:

```

In[ ]:= Show[
  ListPlot[{x, u3}T, PlotTheme → "Detailed", PlotStyle → Black, Frame → True,
    LabelStyle → {12, Black}, FrameLabel → {"x amplitude", "u3 amplitude"},
    AspectRatio → 0.7, PlotLabel → "x(t) vs u3(t) (black dots) and n (blue) and n2 (red)",
    PlotRange → All, ImageSize → 500],
  Plot[{n[xx], n2[xx]}, {xx, -7, 7}, PlotStyle → {Blue, Red}]

```

Out[]:=



The fitted function is similar to the actual Hammerstein system nonlinearity.

Re-predict internal signal, $u_4(t)$

Now that we have the estimate, $n_2(\cdot)$, we can transform the input, $x(t)$, through it to re-predict the internal signal, $u_4(t)$:

```

In[ ]:= u4 = n2[x];

```

Re-estimate the impulse response function

Now that we have a new prediction of the internal signal, $u_4(t)$, we can re-estimate the impulse response function but now between $u_4(t)$ and $y(t)$:

```

In[ ]:= #; uacf = cor[u4, u4,  $\tau_{\text{Max}} / \Delta t$ ];
uyccf = cor[u4, y,  $\tau_{\text{Max}} / \Delta t$ ];

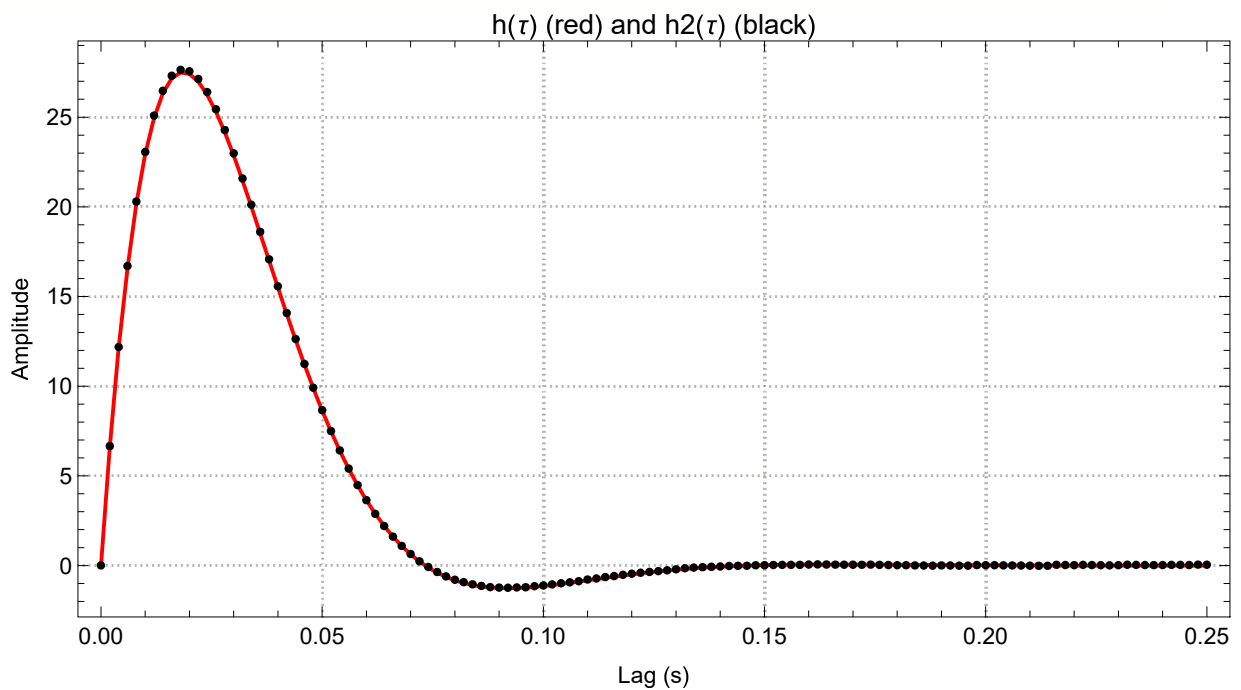
(*toe=ToeplitzMatrix[uacf];
invtoe=Inverse[toe];
hi3=(invtoe.uyccf)/ $\Delta t$ ;*)

{U, S, V} = SingularValueDecomposition[ToeplitzMatrix[uacf]];
threshold = 0.1 Max[S]; (*Keep only significant singular values*)
Sinv = DiagonalMatrix[1 / Max[#, threshold] & /@ Diagonal[S]];
hi3 =  $\frac{1}{\Delta t}$  V.Sinv.Transpose[U].uyccf;

ListPlot[{{ $\tau_i$ , hi}T, { $\tau_i$ , hi3}T}, Joined → {True, False},
PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
LabelStyle → {12, Black}, FrameLabel → {"Lag (s)", "Amplitude"}, AspectRatio → 0.5,
PlotLabel → "h( $\tau$ ) (red) and h2( $\tau$ ) (black)", PlotRange → All, ImageSize → Automatic → 600]

```

Out[]:=



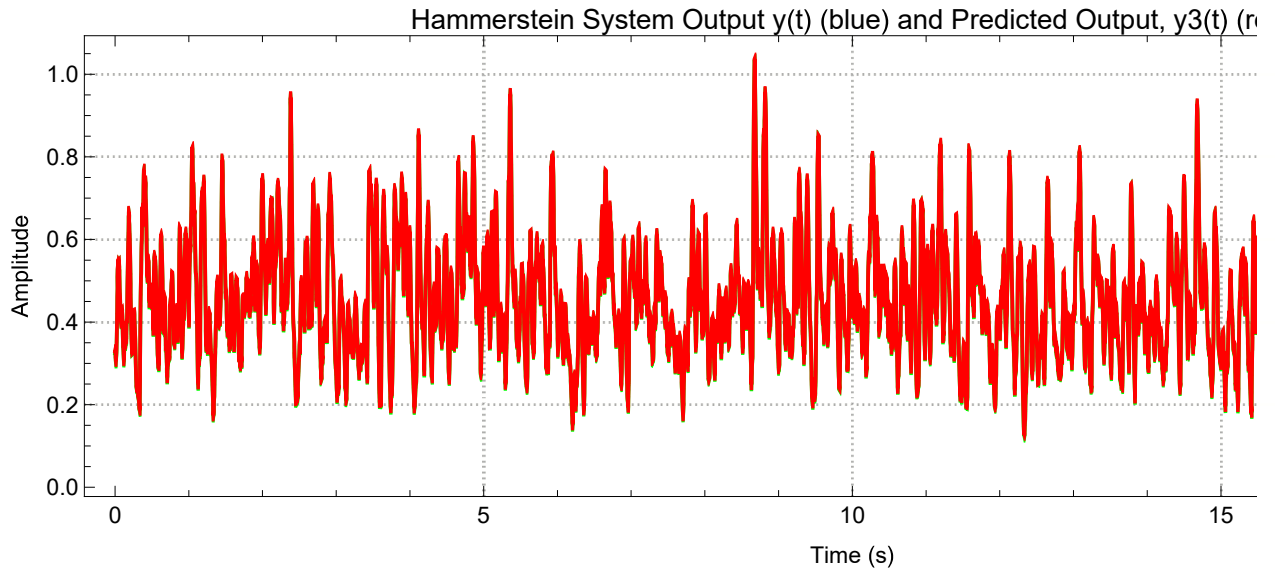
And we can now generate a new output prediction:

```

In[ ]:= ■; y3 = Δt ListConvolve[hi3, u4, {1, 1}] ;
ListLinePlot[{{ti, y}^T, {ti, y3}^T}, PlotTheme → "Detailed",
  PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
  FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3,
  PlotLabel → "Hammerstein System Output y(t) (blue) and Predicted Output, y3(t) (red)",
  ImageSize → Automatic → 800]

```

Out[]:=



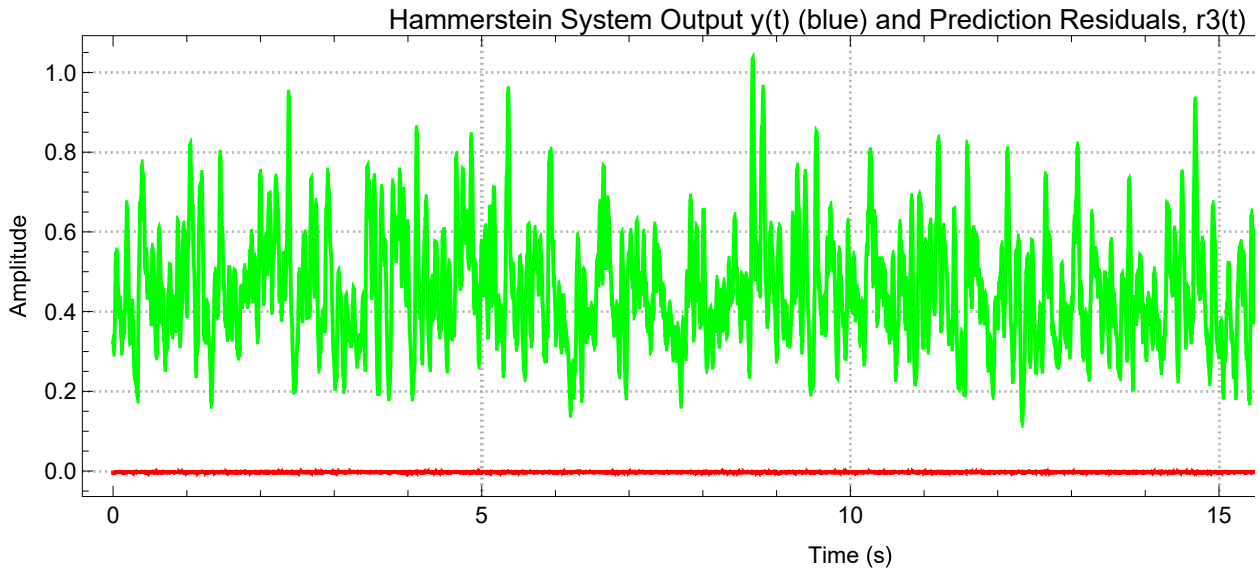
The prediction is so good it is difficult to see the difference between $y(t)$ and $y_3(t)$. So let's calculate the residuals, $r_3(t)$:

```

In[ ]:= ▯; r3 = y - y3;
ListLinePlot[{{ti, y}^T, {ti, r3}^T}, PlotTheme → "Detailed",
  PlotRange → All, PlotStyle → {Blue, Red}, Frame → True, LabelStyle → {12, Black},
  FrameLabel → {"Time (s)", "Amplitude"}, AspectRatio → 0.3, PlotLabel →
    "Hammerstein System Output y(t) (blue) and Prediction Residuals, r3(t) (red)",
  ImageSize → Automatic → 800]

```

Out[]=



```

In[ ]:= ▯; vaf3 = 100.0 (1 - Variance[r3] / Variance[y])

```

Out[]=

99.9991

This is a tad higher than our previous VAF, but we have basically accounted for all the output variance.

```

In[ ]:= ▯; {vaf1, vaf2, vaf3}

```

Out[]=

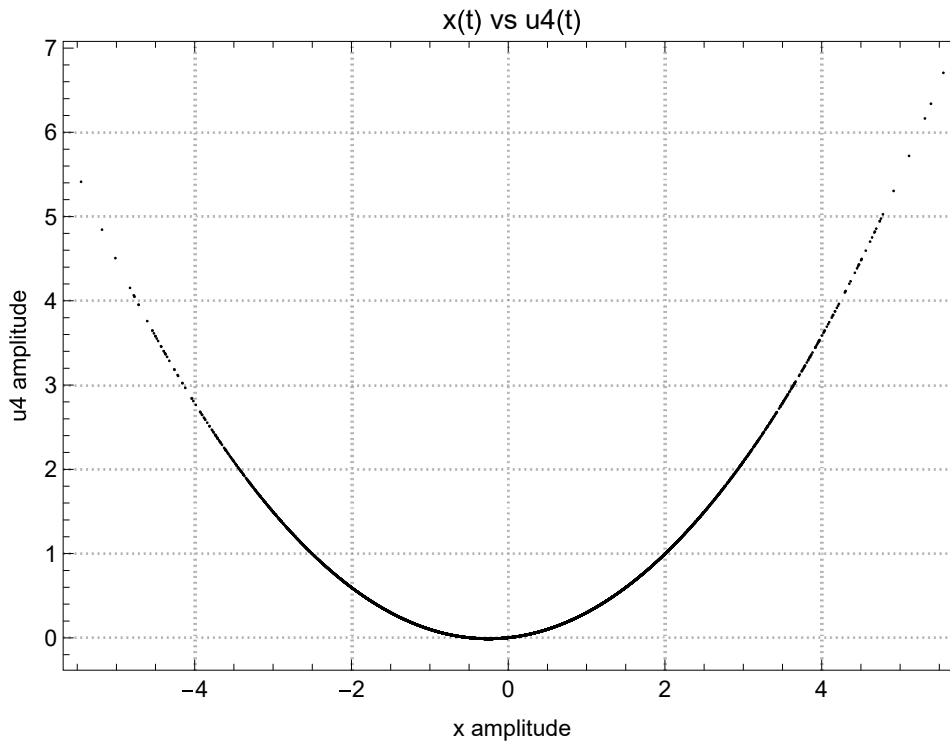
{7.48187, 99.9648, 99.9991}

Re-predict $n(\cdot)$

Now that we have a new estimate, $u_4(t)$, we can predict the form of the static nonlinearity by simply cross-plotting x vs u_4 :

```
In[ ]:= ▯; ListPlot[{x, u4}^T, PlotTheme → "Detailed", PlotStyle → {Red, Black}, Frame → True,
  LabelStyle → {12, Black}, FrameLabel → {"x amplitude", "u4 amplitude"},
  AspectRatio → 0.7, PlotLabel → "x(t) vs u4(t)", PlotRange → All, ImageSize → 500]
```

```
Out[ ]:=
```



Now we have a really low spread in the points.

So lets re-estimate the four parameters of a saturation nonlinearity model using Levenberg - Marquardt nonlinear minimization technique to fit to the $x(t)$ vs $u4(t)$ cross - plot above to give a new estimate $n3(\cdot)$:

```
In[ ]:= ▯;
n3 = LinearModelFit[{x, u4}^T, {1, xx, xx^2, xx^3, xx^4, xx^5, xx^6, xx^7, xx^8, xx^9}, xx] ["Function"];
```

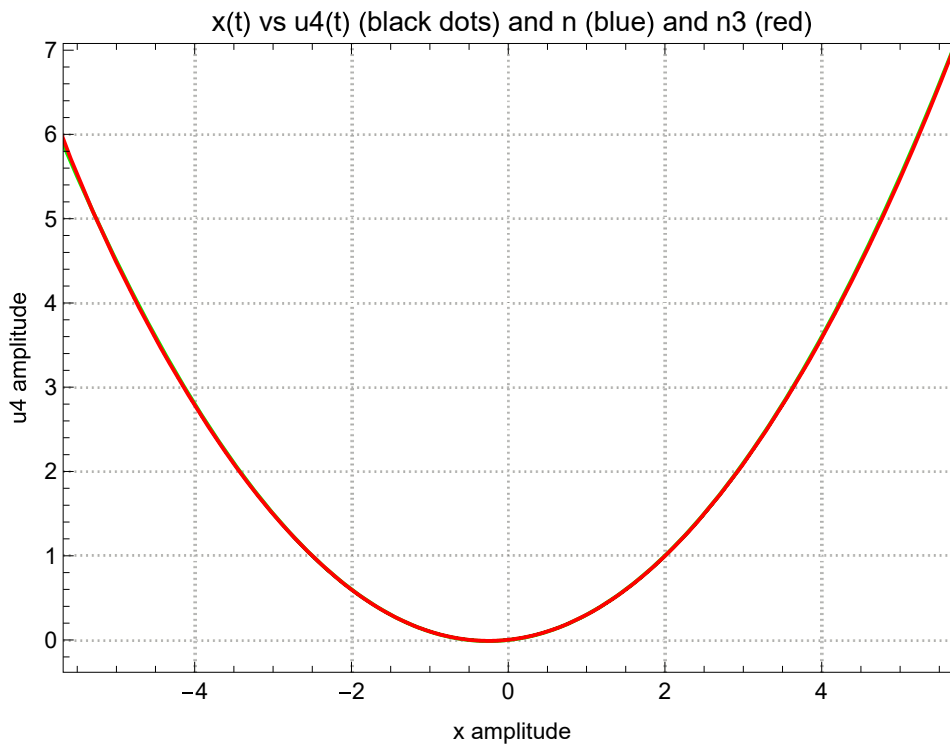
Lets plot this estimate, $n3(\cdot)$, of the nonlinearity together with the cross-plot of $x(t)$ vs $u3(t)$:


```

In[ ]:= ▣; Show[
  ListPlot[{x, u4}^T, PlotTheme → "Detailed", PlotStyle → Black, Frame → True,
    LabelStyle → {12, Black}, FrameLabel → {"x amplitude", "u4 amplitude"},
    AspectRatio → 0.7, PlotLabel → "x(t) vs u4(t) (black dots) and n (blue) and n3 (red)",
    PlotRange → All, ImageSize → 500],
  Plot[{n[xx], n3[xx]}, {xx, -7, 7}, PlotStyle → {Blue, Red}]

```

Out[]:=



Parametric models

Now that we have a good nonparametric model for the impulse response function it is appropriate to go one more step and generate a parametric (equation based) model for it.

Parametric impulse response function model

By observation of the shape of $h_2(\tau)$ it looks as though it might be well approximated (fitted) by a second-order low-pass underdamped impulse response function.

$$In[]:= \blacksquare; h[\tau_, \alpha_, \omega_, \xi_] := \frac{e^{-\xi \tau \omega} \alpha \omega \operatorname{Sinh}\left[\sqrt{-1 + \xi^2} \tau \omega\right]}{\sqrt{-1 + \xi^2}}$$

We start by defining an objective function to be minimized. Lets find the 3 parameters of the parametric second order low-pass underdamped impulse response function which minimize the sum of squared differences between it and the best nonparametric impulse response function model we determined, namely, $h_2(\tau)$.

```
In[ ]:= ■;
      rmsError[α_, ω_, ξ_] :=
        With[{n = Length[hi2]}, Abs[Sqrt[(hi2 - h[τi, α, ω, ξ]) . (hi2 - h[τi, α, ω, ξ])] / (n - 2)]]]
```

From inspection of $h_2(\tau)$ above lets estimate that the natural frequency will be near 7 Hz. Which in rad/s is :

```
In[ ]:= ■; ωω = 2 π 7.0;
```

The system is clearly only slightly underdamped (i.e., a little less than 1.0) so lets set it to be:

```
In[ ]:= ■; ξξ = 0.8;
```

The static gain (i.e., overall gain) is the area under the impulse response function which we forced to be 1, so lets set it to be:

```
In[ ]:= ■; αα = 1.0
```

```
Out[ ]:=
1.
```

```
In[ ]:= ■; rmsError[αα, ωω, ξξ] (* rms error before nonlinear least squares fit *)
```

```
Out[ ]:=
5.87871
```

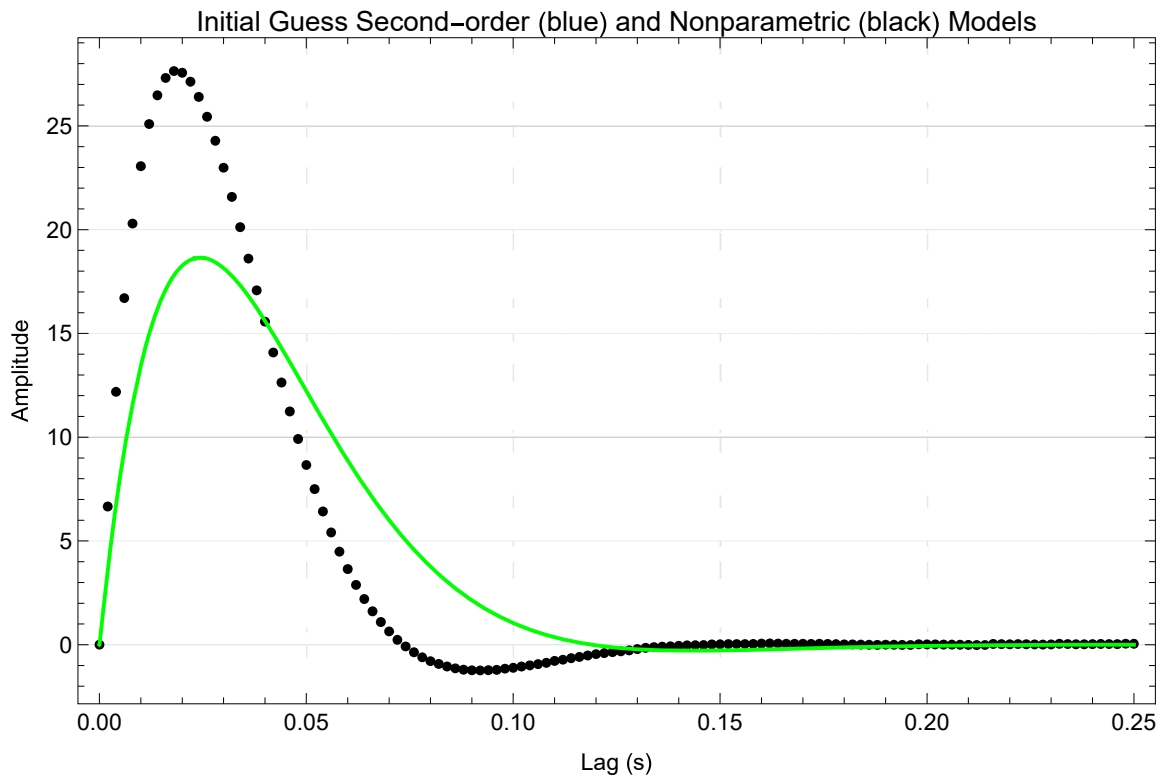
Now lets plot the parametric impulse response function with these estimates together with the nonparametric impulse response function, $h_3(\tau)$:

```

In[ ]:= ■; Show[
  ListPlot[{ $\tau_i$ ,  $hi_3$ }T, PlotRange → All, Frame → True, PlotStyle → Black,
    LabelStyle → {12, Black}, GridLines → Automatic, FrameLabel → {"Lag (s)", "Amplitude"},
    PlotLabel → "Initial Guess Second-order (blue) and Nonparametric (black) Models",
    PerformanceGoal → "Quality", ImageSize → 600],
  Plot[h[ $\tau$ ,  $\alpha\alpha$ ,  $\omega\omega$ ,  $\xi\xi$ ], { $\tau$ , 0,  $\tau_{Max}$ }, PlotStyle → Blue,
    PlotRange → All, PlotPoints → 1000, PerformanceGoal → "Quality"]]

```

Out[]=



Our initial estimates are clearly off. Lets estimate the four parameters using our trusty Levenberg-Marquardt nonlinear minimization technique:

```

In[ ]:= ■;
nlm = NonlinearModelFit[{ $\tau_i$ ,  $hi_3$ }T, h[ $\tau$ ,  $\alpha$ ,  $\omega$ ,  $\xi$ ],
  {{ $\alpha$ ,  $\alpha\alpha$ }, { $\omega$ ,  $\omega\omega$ }, { $\xi$ ,  $\xi\xi$ }},  $\tau$ , Method → "LevenbergMarquardt"];
nlm["BestFitParameters"]
{ $\alpha$ ,  $\omega\omega$ ,  $\xi\xi$ } = { $\alpha$ ,  $\omega$ ,  $\xi$ } /. nlm["BestFitParameters"];

```

NonlinearModelFit: The step size in the search has become less than the tolerance prescribed by the PrecisionGoal option, but the gradient is larger than the tolerance specified by the AccuracyGoal option. There is a possibility that the method has stalled at a point that is not a local minimum.

Out[]=

```
{ $\alpha \rightarrow 1.00565$ ,  $\omega \rightarrow 60.0697$ ,  $\xi \rightarrow 0.701767$ }
```

```

In[ ]:= ■; rmsError[ $\alpha$ ,  $\omega\omega$ ,  $\xi\xi$ ] (* rms error after nonlinear least squares fit *)

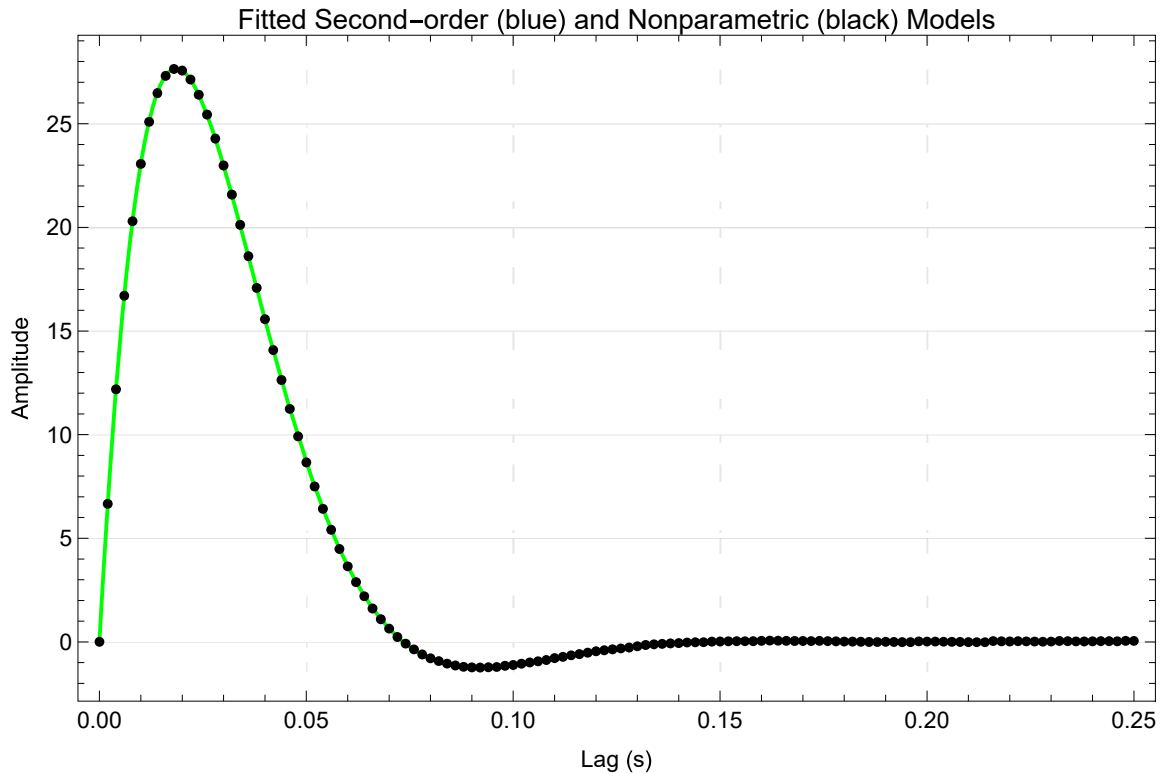
```

Out[]=

```
7.63565
```

```
In[ ]:= ■; Show[
  Plot[h[ $\tau$ ,  $\alpha\alpha$ ,  $\omega\omega$ ,  $\xi\xi$ ], { $\tau$ , 0,  $\tau_{\text{Max}}$ }, PlotRange → All, Frame → True, PlotStyle → Blue,
  LabelStyle → {12, Black}, GridLines → Automatic, FrameLabel → {"Lag (s)", "Amplitude"},
  PlotLabel → "Fitted Second-order (blue) and Nonparametric (black) Models",
  PerformanceGoal → "Quality", ImageSize → 600],
  ListPlot[{ $\tau_i$ , hi3}^T, PlotStyle → Black, PlotRange → All, PerformanceGoal → "Quality"]]
```

Out[]=



Nice! Clearly we have an excellent fit and there is no point going to a higher order model.

The original “true” values for the static gain, natural frequency and damping were:

```
In[ ]:= {1, 60, 0.7}
```

Out[]=

```
{1, 60, 0.7}
```

The estimated values are

```
In[ ]:= ■; { $\alpha\alpha$ ,  $\omega\omega$ ,  $\xi\xi$ }
```

Out[]=

```
{1.00565, 60.0697, 0.701767}
```

These are almost exactly the same. So we have identified the parametric dynamic linear component of the Hammerstein system.

Parametric static nonlinear element model

It remains for us to find a parametric model for the static nonlinearity. The last estimate of the static nonlinearity was n3. Mathematica provides a function called FindFormula[] which attempts to discover

static models which fit a data set. It takes a while because it is trying a huge number of candidate models:

```
In[- ]:= ■; fit = FindFormula[{x, u4}^T, uu, 10, All]
```

```
Out[- ]:=
```

	Score	Error	Complexity
$0.00222795 + 0.0993721 uu + 0.15$	62.1839	0.00000000000000000000000000852838	63
$0.1 uu + 0.2 uu^2$	11.249	0.0000126362	13
$-0.338339 + 0.66128 \text{Abs}[uu]$	2.5889	0.0735624	9
$0.502129 \text{Abs}[uu]$	2.13816	0.116254	6
$0.715705 + 0.594203 \text{Tan}[\text{Cos}[3.]]$	1.82446	0.156188	14
$0.765714 + 0.44653 \text{Tan}[\text{Cos}[3.2]]$	1.64655	0.1866	14
$0.384118 + 0.44653 \text{Tan}[\text{Cos}[3.2]]$	1.38242	0.236388	26
0.453831	0.832298	0.434048	1
0	0.443968	0.64001	1
$\text{Tan}[\text{Cos}[3.57558 + uu]]$	-0.0874527	1.07147	8

This discovery function returns suitable static models together with a “Score” and a measure of “Complexity”. A “good” model is one which has a high Score (a good fit) but a low level of complexity under the assumption that simplicity is important. Using these two criteria we choose the $p1 uu + p2 uu^2$ model. This model form is exactly the static nonlinearity used in the Hammerstein model. Furthermore the two parameters of this model are $p1 = 0.1$ and $p2 = 0.1$ are has a value near to 10 which is the same as that used when the Wiener system was specified. In other words we have identified the static nonlinearity.

FindFormula does a quick fit and test of a wide range of functions. Lets redo the fit:

```
In[- ]:= ■; LinearModelFit[{x, u4}^T, {1 xx, xx^2}, xx]
```

```
Out[- ]:=
```

```
FittedModel[0.00186 + 0.0995 xx + 0.199 xx^2]
```

This model is almost exactly the same static nonlinearity as used in our simulation $u = 0.1 x + 0.2 x^2$.

In conclusion we have managed to identify both the static nonlinear part and the linear dynamic part of the the Hammerstein system.

The Hammerstein system has been identified!!